

Graph-Enhanced Research Paper Recommendation System

Using Embedding-Based Clustering

Kasparas Marcinkas (S235116)

S Taimur Hassan (S250112)

M Abbas Khan (S250145)

David Brin (S251557)

Gabriel Loayza (S252608)

Group Number: 36

02807 Computational Tools for Data Science

DTU (Technical University of Denmark)

December 2025

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Project Goals	1
2	Data	1
2.1	Dataset Source	1
2.2	Schema and Selection	1
2.3	Preprocessing Steps	2
3	Methodology Overview	2
3.1	End-to-End Pipeline Description	2
3.2	Course Topic 1: K-Means Clustering	3
3.3	Course Topic 2: Graph Clustering	3
3.4	External Method: USE	4
3.5	External Method: KNN/HNSW	4
3.6	Threshold Selection	4
4	Results and Evaluation	4
4.1	Descriptive Statistics	4
4.2	Distance Histograms and Threshold	5
4.3	Graph and Clustering Outcomes	5
4.4	Success of the Method	5
5	Discussion and Limitations	6
5.1	Embedding Model	6
5.2	Compute and Storage	6
6	Conclusion and Future Work	6
A	Jupyter Notebook Reference	6
B	Code Instructions	6
C	Contribution Breakdown	7
D	Additional Visualizations	7
E	Declaration of Generative AI Use	10

1. Introduction

1.1 Motivation

Researchers often spend most of their time searching for papers related to their subject to learn about the current landscape of their research or identify other researchers to collaborate with or learn from. The most common method to do so is to use search engines or query databases to find papers regarding a subject. This method can be slow, inefficient, and sometimes ineffective. Recommendation systems have been proposed, but running recommendation systems on the growing database of research papers is increasingly time and compute-intensive. The method we propose turns time complexity into memory complexity for ease of access and quick computation. Researchers should be able to have fast access to communities of like-minded individuals and similar papers.

1.2 Problem Statement

Since large datasets can cost a lot of time or computation to find recommendations, a graph of similar papers with links between papers would create direct links between papers and reduce query time. Further, creating clusters within the graph creates a community of similar papers and brings together researchers working on similar topics. A similarity algorithm, like a recommendation system, can be run on the dataset first to identify similar papers and construct a network using vectorized sentiment encodings of the paper's description, which can then be optimized and clustered to create communities.

1.3 Project Goals

The main goal of this project was to have a quick research paper recommendation and community-building tool that can quickly and easily return a list of similar papers given a query of a paper's title.

2. Data

2.1 Dataset Source

The data is sourced from the arXiv dataset, which is a collection of research papers. In their own words, the database provides:

“open access to scholarly articles, from the vast branches of physics to the many subdisciplines of computer science to everything in between, including math, statistics, electrical engineering, quantitative biology, and economics. This rich corpus of information offers significant, but sometimes overwhelming depth.”

That overwhelming depth is what we are attempting to overcome.

2.2 Schema and Selection

The data is a snapshot of a larger dataset of papers, which makes it possible for us to work on it without larger compute power and storage. The data is stored as JSON entries referencing the scientific paper with some metadata.

Here is one example entry (truncated for readability):

```
"root": {
  "id": "0704.0001",
  "submitter": "Pavel Nadolsky",
  "authors": "C. Balázs, E. L. Berger,
             P. M. Nadolsky, C.-P. Yuan",
  "title": "Calculation of prompt diphoton
            cross sections at Tevatron and LHC energies",
  "comments": "37 pages, 15 figures;
              published version",
  "journal-ref": "Phys.Rev.D76:013009,2007",
  "doi": "10.1103/PhysRevD.76.013009",
  "categories": "hep-ph",
  "abstract": "A fully differential calculation
              in perturbative quantum chromodynamics ..."
}
```

2.3 Preprocessing Steps

Preprocessing is intentionally light. We strip whitespace from titles and abstracts, drop rows with invalid years or empty abstracts, and compute abstract length and simple word count for exploratory plots.

The abstracts with fewer than 200 words and papers over 10 years old are filtered out before embedding to ensure that only abstracts containing significant semantic information are retained.

For scalability, abstracts are processed in shards of approximately 25,000 papers. For each shard, we compute embeddings in smaller batches, store the resulting 512-dimensional vectors as `emb_XXXXX.npy` (float16), and write a matching `meta_XXXXX.tsv` file containing year and title. These shards are later loaded with memory mapping, allowing us to iterate over embeddings in fixed-size chunks when building the index and the graph.

3. Methodology Overview

3.1 End-to-End Pipeline Description

The data (JSON inputs) goes through the following modules, in order:

- Exploratory Data Analysis (filtering and metadata)
- Sentiment analysis and vector embeddings
- K-means clustering
- KNN/HNSW-based graph construction
- Graph clustering
- Threshold selector

- Community querying

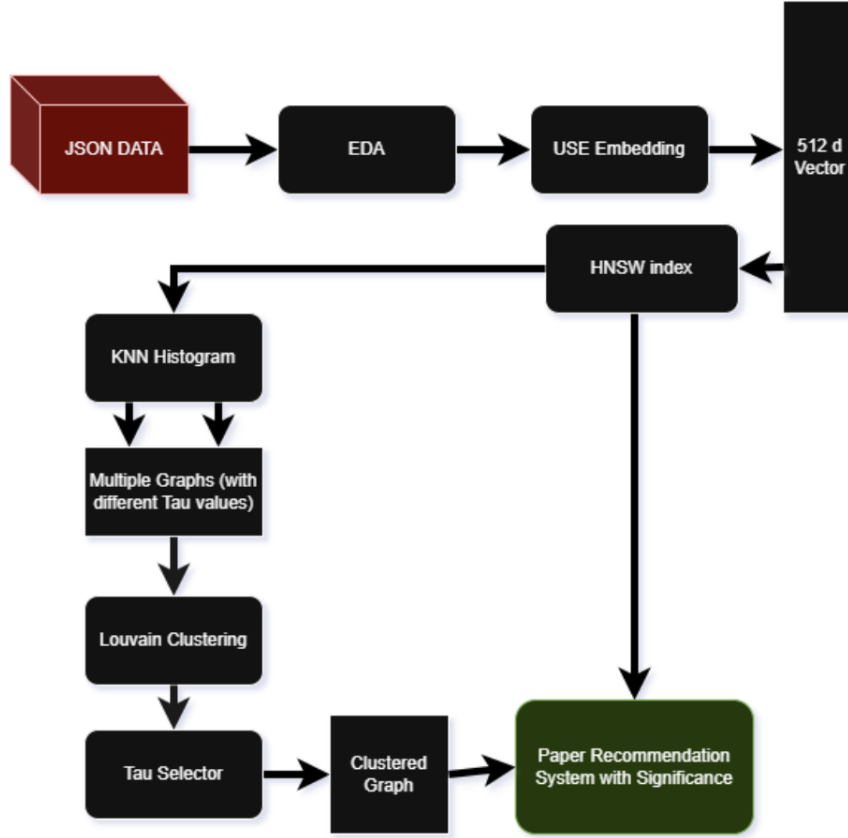


Figure 1: Visual representation of the end-to-end recommendation pipeline, from raw JSON ingestion to community querying.

3.2 Course Topic 1: K-Means Clustering

K-Means clustering is used to partition the embedding space into k clusters. We use scikit-learn’s MiniBatchKMeans, which updates cluster centres on small batches and can handle large datasets.

We experimented with several k values on subsets and inspected cluster sizes and sample titles. We selected a k where clusters were neither extremely imbalanced nor too fine-grained, and where manually inspected clusters showed clear topical similarity (for example, many papers on graph neural networks in one cluster). The clustering was used before implementing a KNN (later improved to ANN) classifier to identify neighbors for graph creation.

3.3 Course Topic 2: Graph Clustering

From the thresholded edge list (list of neighbors), we built a weighted, undirected graph whose nodes are papers and whose edges represent strong semantic similarity. On this graph, we apply a modularity-based community detection algorithm, which repeatedly groups nodes into communities to maximize modularity. The result is a partition of the graph into communities that have many internal edges and relatively fewer edges to other communities.

The algorithm used is the Louvain algorithm. This algorithm decides where modularity is most optimized on a local level, which allows for better community creation by maximizing

modularity and bringing together communities of papers that need aggregation.

Once the communities are created, we implement querying functionality that returns the community in which the queried paper resides, giving the user access to all papers related to the one they are interested in.

3.4 External Method: USE

A Universal Sentence Encoder (USE), made by Google (from TensorHub), was used as the NLP module that encodes each abstract into an embedding vector. This vectorization allows for computational tools like clustering, classifiers, and, therefore, graph creation. It enables computational tools by encoding semantic information as high-dimensional numerical data.

3.5 External Method: KNN/HNSW

To find related papers and create the graph, we originally ran a KNN on the clustered embeddings to classify the nearest semantic neighbors and construct edges between all neighbors found. A KNN works by calculating the distance to all data points and returning the smallest values. This gets extremely computationally intensive with many data points and running the model on every point (which can be helped slightly with clustering). We planned to run this several times to tune hyperparameters based on connectivity statistics (like average degree and distance), which illuminated the fact that our method was too inefficient.

To improve efficiency, we looked to the ANN (Approximate Nearest Neighbor), and more specifically, the HNSW (Hierarchical Navigable Small World) algorithm. This algorithm is a graph-based algorithm that uses a layered architecture to simplify and speed up distance queries, significantly decreasing the time it takes to find the nearest neighbors of a point. So we ended up using a graph to build another graph. The original HNSW index is maintained to rank the final recommended papers within the clusters by the significance of related documents.

3.6 Threshold Selection

Threshold selection was performed by first choosing a threshold by the amount of connections kept, based on a histogram of the distances of the neighbors from KNN queries on the HNSW index. This threshold is then used to create a graph by filtering out each connection that has a distance longer than the threshold. This process is repeated on different values of connections kept to create multiple graphs with different threshold (τ) values. The clustering is then run on each of these graphs and modularity is calculated on the clustered graphs. The graph with the highest score is then kept as the final clustered graph.

4. Results and Evaluation

4.1 Descriptive Statistics

We built multiple graphs by changing the distance threshold τ , which controls how similar two papers must be to connect them.

The final graph at $\tau = 0.27$ includes 148,477 papers, with nearly every paper connected to at least one other. As the threshold increases, the number of edges and the average degree increase, while the number of isolated papers decreases. At $\tau = 0.19$, most papers had no connections.

By $\tau = 0.23$, the average degree increased to 4.3 and isolated nodes dropped below 53%. At $\tau = 0.27$, isolated nodes were almost eliminated. This shows that higher thresholds lead to a more connected and useful graph.

4.2 Distance Histograms and Threshold

To choose a good threshold, we looked at the histogram of distances between paper embeddings. We tried several τ values and checked how they affected graph sparsity. Too low meant very few links and too many disconnected papers. Too high meant noisy links. $\tau = 0.27$ was selected as the sweet spot. It keeps strong semantic links while reducing isolated nodes and enabling useful communities. This balance made it a good choice for downstream clustering and recommendations.

4.3 Graph and Clustering Outcomes

After building graphs at different thresholds, we ran Louvain clustering to find paper communities. At $\tau = 0.19$, we got over 133,000 small communities, most of them single papers. The modularity was very high, but the graph was not useful. As we increased τ , the number of communities dropped and the modularity decreased slightly, which is expected as the graph becomes more connected. By $\tau = 0.23$, we had $\sim 81,000$ communities and a modularity of ~ 0.8 . This shows we started grouping similar papers together. At $\tau = 0.27$, the communities became much larger and more meaningful, helping with better recommendations.

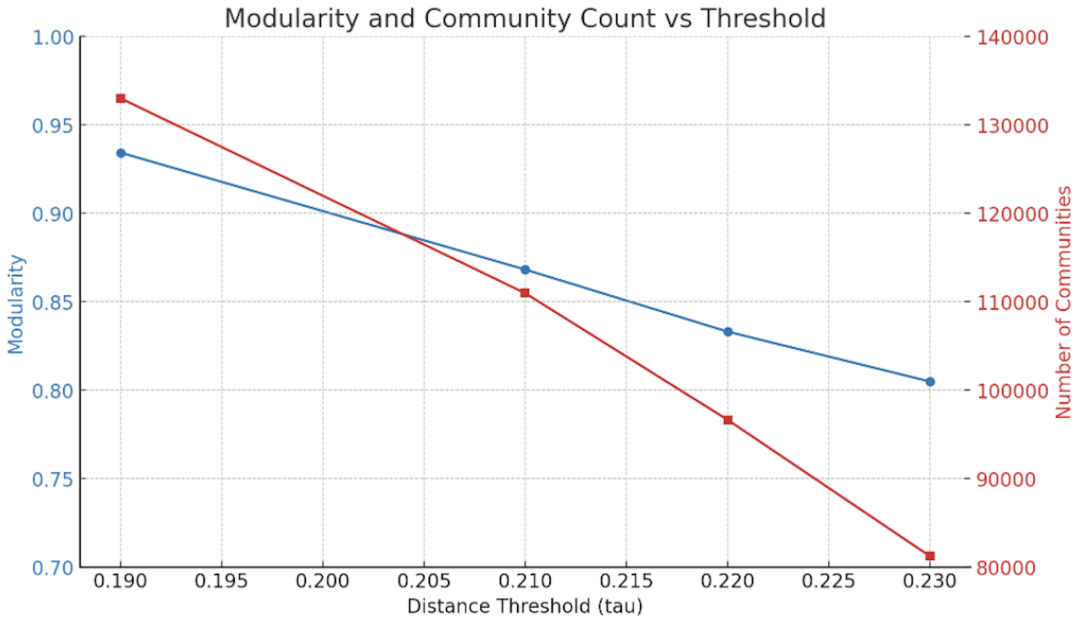


Figure 2: Effect of threshold on modularity (blue, left axis) and number of communities (red, right axis). As τ increases from 0.19 to 0.23, modularity drops from ~ 0.93 to ~ 0.80 , while the number of communities collapses from $\sim 133k$ to $\sim 81k$.

4.4 Success of the Method

Our method performed well in both speed and quality. The use of the HNSW index made the nearest neighbor search fast and scalable. Louvain clustering gave us paper groups that

aligned with real research topics. By shifting computation to the start, we made querying fast at runtime. The final graph allowed us to retrieve similar papers instantly. Although $\tau = 0.19$ had the best modularity, it was not practical. By choosing $\tau = 0.27$, we got better usability with still strong modularity, showing the method worked as planned.

5. Discussion and Limitations

5.1 Embedding Model

The embedding model was Google’s Universal Sentence Encoder, the fourth version from TensorFlow Hub. This model is a Natural Language Processing tool that enables the rest of the computational tools to work on semantic data. Since the rest of the project depends on these encodings, a fault in the embeddings would be a severe limitation, as the data pipeline relies on the quality of sentence embeddings.

5.2 Compute and Storage

The main drawback of this graph-based method of recommending papers is the memory complexity or storage size. The time complexity of querying a paper is significantly decreased in exchange for a large data structure taking up space in memory. We argue that this exchange is worth the cost, as now a computer can run many queries and send out the responses to the ‘customer’ at a much faster rate.

6. Conclusion and Future Work

The semantic graph structure trades compute/time complexity for space/memory complexity, making a faster and more thorough recommendation system and community creation for research papers. Hyperparameters like the threshold can be changed for different use-cases, but for our purposes, we used metrics like modularity and average degree to create a more connected graph that connects more papers to a community. By keeping the HNSW index, the user can filter for significance in the related paper query.

Future work could include creating a graphical user interface to make the graph system more accessible and understandable to a wider range of users. Further, to make use of the quick compute capabilities, adding a dedicated server and an accessible client end, the speedup in queries can be effectively used and significant.

A. Jupyter Notebook Reference

<https://github.com/loayzapre/arxiv-semantic-graph/tree/main>

B. Code Instructions

To run the notebook:

1. Install Python 3.9+ and Jupyter Notebook.
2. Install required libraries:


```
pip install tensorflow glob pandas matplotlib \
numpy pathlib scikit-learn tqdm
```

3. Place the notebook and data in a folder and replace “DIR” variables with the corresponding path.
4. Open a terminal, go to the project folder, and start Jupyter:

```
cd path/to/project
jupyter notebook
```

5. Open `project_demo.ipynb` and select Kernel → Restart & Run All.

The notebook will load data, run the model, and show outputs in place. If errors occur, check that folder paths are correct or install any missing Python packages.

C. Contribution Breakdown

As mandated by the formal requirement, which emphasizes the need for transparency in individual contributions to ensure grading fairness, the group and its members wish to affirm that each member has participated and delivered a commendable and closely equal contribution across the different sections of the project. Should further clarification be required, please feel free to contact any of the group members.

D. Additional Visualizations

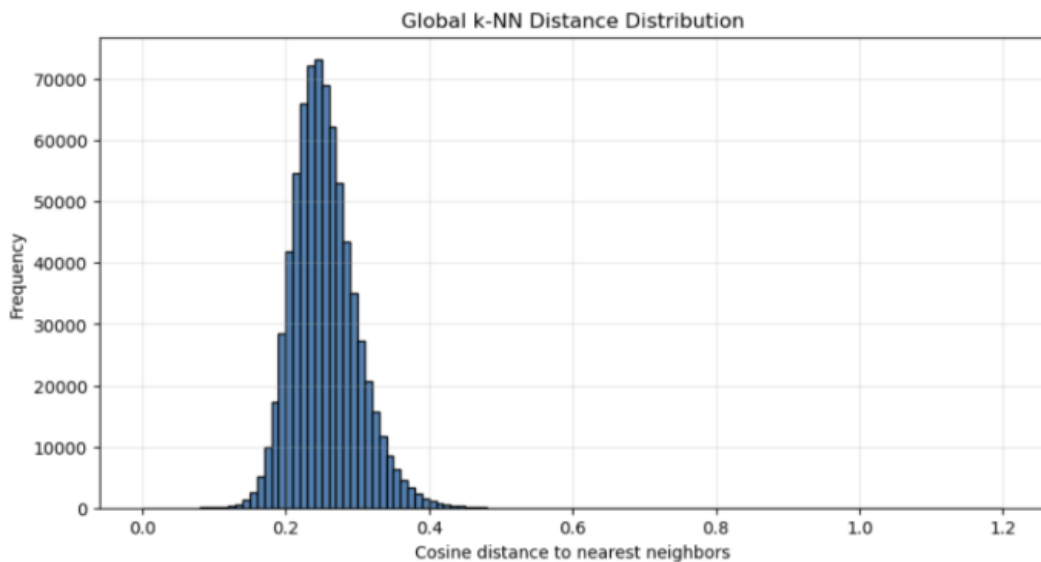


Figure 3: Histogram of nearest-neighbor distances used for threshold selection.

[Graph] Summary of constructed graphs:

	pkeep	tau	nodes	edges	avg_degree	median_degree	isolated_nodes
0	0.05	0.19	148477	25347	0.341427	1.0	129176
1	0.10	0.21	148477	101172	1.362797	2.0	106718
2	0.15	0.22	148477	186608	2.513628	2.0	92680
3	0.25	0.23	148477	321803	4.334719	3.0	77704

Figure 4: The figure highlights the optimal threshold choice that balances having sufficient connections for recommendations while preserving high-quality, semantically coherent clusters

[Louvain] Summary of results:

	pkeep	tau	modularity	num_communities
0	0.05	0.19	0.934340	133006 /h
1	0.10	0.21	0.868229	110964 /h
2	0.15	0.22	0.833143	96650 /h
3	0.25	0.23	0.804924	81225 /h

Figure 5: Graph clustering of papers using the Louvain algorithm.

```

=====
Random query paper
=====
ID   : 105111
Year : 2025
Title : Efficient Transformed Gaussian Process State-Space Models for Non-Stationary High-Dimensional Dynamical Systems
=====

[Recommend] Loaded Louvain partition for 148,477 nodes.
[Recommendations]
- (id=104672, year=2025) ✓ significant
  On the Importance of Error Mitigation for Quantum Computation
  distance=0.1649

- (id=55233, year=2024) ✓ significant
  Photonics-integrated terahertz transmission lines
  distance=0.1852

- (id=67817, year=None) ✓ significant
  rational function fields over ultra-finite fields
  distance=0.1925

- (id=57998, year=None) ✓ significant
  Attention Pooling
  distance=0.1954

- (id=52844, year=2025) ✓ significant
  A Progressive Risk Formulation for Enhanced Deep Learning based Total
  distance=0.1979

[Recommend] Loaded Louvain partition for 148,477 nodes.

```

Figure 6: Example of a graph-based paper recommendation query. A query paper (highlighted) is used to retrieve its most similar neighbors from the semantic graph. The system returns several closely related papers (either shown as connected nodes in the same community or listed by title) that share strong topical similarity with the query.

E. Declaration of Generative AI Use

AI in the form of Large Language Models (LLMs) was used in this project. These tools were mostly used as a formatting tool and a streamlined search engine. For example, once an idea or script had left the exploratory phase and needed to be incorporated into the project, the code required readability and conciseness. Therefore, the code was given to a chatbot (such as Claude or ChatGPT) to identify unused functions and to write comments and explanations about the code. These tools also provided markdown summaries of the project and kept track of data types.

In the future, we would like to see the use of agentic workflows for a more automated project creation.